



FORMAOS

COMPLIANCE, SIMPLIFIED

CTO HANDOVER PACK · 01

Project Handover & Executive Overview

FormaOS — a multi-tenant compliance operations platform. Current production state, what it does, what is being handed over, and the one item standing between today and launch-ready.

PREPARED
FOR
Farhad
Hussain
Incoming CTO

PREPARED
BY
Ejaz
Hussain
Founder

DATE
13 June
2026

CLASSIFICATION
Confidential

What you are inheriting

FormaOS is a production compliance-operations platform for regulated service providers — NDIS, healthcare, aged care, childcare, mental health and financial services. It turns compliance frameworks into day-to-day operational work: controls, evidence, tasks, posture scoring and an immutable audit trail, multi-tenant from the ground up.

221 APP & SITE PAGES across product, marketing, admin	283 API ROUTES REST + server actions	252 DB MIGRATIONS Supabase / Postgres	8 FRAMEWORK PACKS SOC2, ISO, GDPR, HIPAA...
Next.js 16 APP FRAMEWORK React 19 · TypeScript 5	Supabase DATA & AUTH Postgres · RLS · storage	Vercel HOSTING syd1 region · native git	Live PRODUCTION formaos.com.au

The product in one paragraph

An organisation signs up, picks its industry, and FormaOS provisions a tenant workspace pre-loaded with the relevant framework packs. From there the team works a guided first session (create a care plan, add goals, log progress notes, upload evidence, review tasks), and the platform continuously computes a compliance posture from controls and evidence — surfaced as an executive dashboard, exportable audit packs, and a tamper-evident activity trail. Billing, trials and entitlements run through Stripe; background work runs on Trigger.dev; everything is observable through Sentry, PostHog and OpenTelemetry.

Industries & frameworks supported

INDUSTRY VERTICAL	REPRESENTATIVE FRAMEWORKS SHIPPED AS PACKS
NDIS / disability	NDIS Practice Standards, incident (SIRS) tracking, worker screening
Healthcare	NSQHS, AHPRA, RACGP, HIPAA
Aged care	Aged Care Quality Standards, star ratings
Childcare	NQF / educator credentials
Mental health	Mental-health-AU pack, consumer care plans
Financial services	SOC 2, ISO 27001, PCI-DSS, breach register
Cross-industry	GDPR, CIS Controls, NIST CSF

Where the project stands today

STATUS · LIVE IN PRODUCTION

The application is deployed and serving traffic at formaos.com.au / app.formaos.com.au (Vercel, Sydney region). The public surface, authentication, and the full authenticated product are reachable; 382 internal app links validated with zero broken links.

Shipped in the most recent cycle (this week)

The product UI was elevated to a restrained, monochrome enterprise system, and the new-user experience was de-cluttered end to end. Seven changes shipped and are live:

#	CHANGE	STATE
248	Monochrome enterprise UI sweep (218 files) — consolidated status palette, removed decorative glass/ornaments	LIVE
249	Fixed onboarding bug — completion toasts re-firing on every page load (tenant-table registry omission)	LIVE
250	Focused first-session dashboard — guide + live tiles only, instead of ~10 empty blocks	LIVE
251	Compliance page — setup panel for empty orgs, two-block register	LIVE
252	One onboarding list at a time — removed competing checklists	LIVE
253	Settings — single-column hub, every live value rendered once	LIVE
254	Raised auth bootstrap function timeout (partial mitigation of the blocker below)	LIVE

The one launch blocker

BLOCKER · NEW-USER PROVISIONING TIMES OUT

On a brand-new signup, the first login calls `/api/auth/bootstrap`, which provisions the tenant workspace via ~30 sequential database statements. Against the production database under load this exceeds the serverless function timeout and returns a **504** — i.e. the very first-time-user can hit a dead first login. Existing users are unaffected (their path short-circuits). This is a database-performance problem, not a UI one, and is the single item I would resolve before calling the app 'launch-ready'. Full root-cause and fix options are in **Document 04 — Project Plan & Roadmap**.

What you are reading

DOCUMENT	PURPOSE	FOR
01 · Handover Overview	This document — the snapshot, status and the blocker	Orientation
02 · Developer Documentation	Architecture, stack, repo map, local setup, CI/CD, conventions	Engineering
03 · Workflow Mapping	The core execution flows: request lifecycle, auth, onboarding, compliance, billing, release	Engineering / product
04 · Project Plan & Roadmap	Status, the blocker fix, backlog, risk register, phased roadmap, a 30/60/90 plan	Leadership / planning

DEEPER DETAIL IN THE REPO

Beyond this pack, the repository carries the long-form engineering handover: `docs/HANDOVER/` (full secrets inventory, services & access checklist, monthly cost surface, and decision-history ADRs) and operational runbooks at the repo root — `ONCALL.md` , `RUNBOOKS.md` , `SENTRY-ALERTS-GUIDE.md` , and `docs/operations/pitr-restore-runbook.md` .

Access & accounts to transfer

I've grouped these by system. I'll hand each one over and confirm you have owner or admin access on every one before we wrap up.

SYSTEM	WHAT IT HOLDS	ACTION
GitHub (FormaOS repo)	Source, CI workflows, PR history	Add as admin / transfer ownership
Vercel (forma-os project)	Production hosting, env vars, deploy history, domains	Add to team as owner
Supabase (prod project)	Database, auth, storage, RLS, migrations	Add as organization owner
Stripe	Subscriptions, plans, webhooks, billing	Add team member / admin
Resend	Transactional email — auth, billing, password resets	Add as admin (API key + domains)
Upstash Redis	Rate limiting, cache, soft job queue	Add as workspace owner
Sentry & PostHog	Error monitoring & product analytics	Add as admin
Secret manager (1Password)	Prod service-role & encryption keys — source of truth outside Vercel	Grant vault access (day 1)
Google Workspace	Founder email + identity for formaos.com.au	Add as super-admin / transfer
Domain & DNS registrar	formaos.com.au DNS (A/MX/SPF/DKIM/DMARC)	Grant access / transfer
Trigger.dev	Background jobs / workflow runs	Add to project

NOTE

Secrets themselves (API keys, service-role keys, Stripe keys) live in Vercel environment variables and the local `.env.local` — rotate any that I've shared informally as part of the handover, and re-issue them under your accounts.



FORMAOS
— COMPLIANCE, SIMPLIFIED —

CTO HANDOVER PACK · 02

Developer Documentation

Architecture, technology stack, repository map, local setup, testing, CI/CD and the conventions you'll need to work safely in the codebase.

PREPARED
FOR
**Farhad
Hussain**
Incoming CTO

PREPARED
BY
**Ejaz
Hussain**
Founder

DATE
**13 June
2026**

CLASSIFICATION
Confidential

Stack & key dependencies

A TypeScript monorepo on the Next.js App Router. Server components and server actions do the heavy lifting; the database is the source of truth with row-level security enforced per tenant.

LAYER	TECHNOLOGY	VERSION	ROLE
Framework	Next.js (App Router)	16.1	SSR, server actions, routing
UI	React + TypeScript	19.2 / 5.9	Components, type safety
Styling	Tailwind CSS	3.4	Design tokens, theming
Data / Auth	Supabase (Postgres)	2.98	DB, auth, storage, RLS
Billing	Stripe	22.1	Subscriptions, trials, webhooks
Background jobs	Trigger.dev	—	Async workflows, exports
Rate limit / cache	Upstash Redis	—	Throttling, operational cache
Validation	Zod	4.3	Runtime schema validation
Observability	Sentry · PostHog · OTel	10.4 / 1.36	Errors, analytics, tracing
Testing	Jest · Playwright	30 / 1.58	Unit, E2E, a11y, visual

Repository map

```
app/ # Next.js App Router
  (marketing)/ # public site (own theme)
  app/ # authed product — 47 feature areas
  admin/ # platform / founder console
  api/ # 283 REST routes + webhooks
  auth/ onboarding/ # sign-in, MFA, first-run
lib/ # domain services (the business logic)
  supabase/ # clients incl. org-scoped tenant wrapper
  compliance/ frameworks/ # controls, graph, scoring
  billing/ auth/ provisioning/ # Stripe, sessions, tenant setup
  care/ evidence/ export/ admin/ # domain modules
components/ # shared React UI (ui/ primitives + features)
supabase/migrations/ # 252 SQL migrations (schema + RLS history)
framework-packs/ # 8 JSON control libraries
trigger/ # background jobs
e2e/ __tests__/ tests/ # Playwright, unit, integration
.github/workflows/ # 12 CI pipelines
```

WHERE THE LOGIC LIVES

Treat `lib/` as the heart of the system. Routes and components are thin; the real domain rules — tenancy, compliance scoring, billing, provisioning — live in `lib/` services and are unit-tested there.

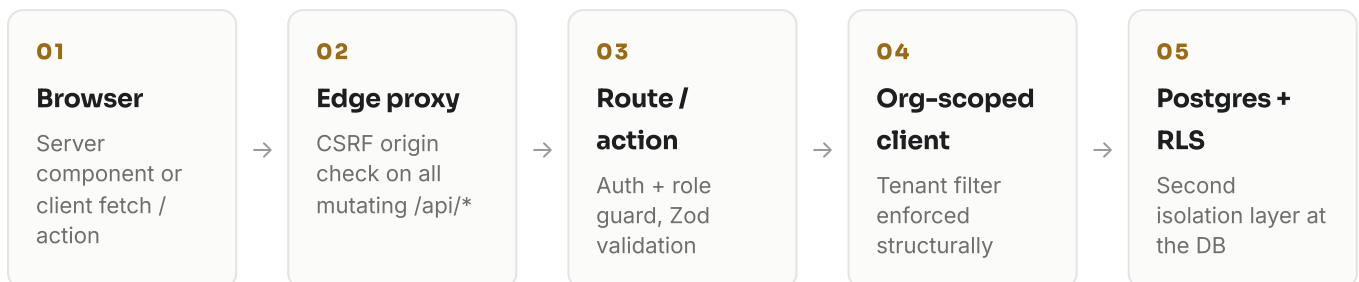
Multi-tenancy & the security model

FormaOS is multi-tenant: every organisation's data lives in shared tables keyed by `organization_id`, isolated by Postgres **Row-Level Security**. Because some server work uses the Supabase service-role key (which bypasses RLS), the codebase adds a second structural guard:

CRITICAL CONVENTION · THE ORG-SCOPED CLIENT

`createSupabaseOrgClient(orgId)` (in `lib/supabase/org-scoped.ts`) wraps the admin client so every query is automatically filtered/stamped by the tenant column, and **any table not registered in `TENANT_TABLE_SCOPES` throws at runtime**. This is deliberate: it forces an explicit decision for each table rather than relying on 800+ hand-written filters. When you add a tenant table, you must register it here — a recently-fixed onboarding bug was caused by exactly this omission.

Request path (every authenticated mutation)



Auth & access

- Supabase Auth with email, Google OAuth, SAML/SSO, and MFA (TOTP + backup codes).
- Role-aware access (owner / admin / member / viewer) plus delegated platform-admin roles in the admin console.
- Sessions are cookie-based with a tracked-session layer; high-risk admin actions are reason-gated and approval-gated.

Key subsystems at a glance

SUBSYSTEM	LIVES IN	WHAT TO KNOW
Compliance engine	lib/compliance*, framework-packs/	Framework packs (JSON) define controls; evidence + evaluations roll up to a posture score and a node-wire compliance graph.
Provisioning	lib/provisioning/	Tenant + user setup on first login (the bootstrap path — see the blocker).
Billing	lib/billing/	Stripe subscriptions, trials, entitlements; webhook-driven state. Plans gate features via entitlements.
Evidence & vault	lib/evidence/, app/app/vault	File uploads mapped to controls; what auditors sample.
Admin console	app/admin, lib/admin	Founder + delegated admins; customer-360, security ops, releases, audit reads.
Background jobs	trigger/	Data retention, purges, audit-chain anchoring, health snapshots (cron + Trigger.dev).
Observability	sentry.*, lib/monitoring	Sentry errors, PostHog analytics, OpenTelemetry traces.
Backups & DR	Supabase PITR · docs/operations/	7-day point-in-time recovery; RPO 60 min / RTO 4 h; CI-gated monthly restore drills logged to <code>restore_test_runs</code> .

DATA RESIDENCY

Application compute and edge run in **Vercel** `syd1` (**Sydney**); the database, authentication and file storage run in **Supabase** `ap-southeast-1` (**Singapore**). So customer data is served from AU but stored in SG today — worth stating plainly for AU clients and auditors (NDIS, health). Migrating the database to an AU region is the clean answer if in-country data residency becomes a procurement requirement.

Local setup & testing

Run it locally

```
# 1 • install
npm ci

# 2 • environment (copy and fill secrets)
cp .env.example .env.local

# 3 • dev server
npm run dev

# 4 • typecheck / lint / build
npm run typecheck • npm run lint • npm run build
```

Test suites

COMMAND	SUITE	NOTES
npm run test:coverage	Jest unit + integration	The authoritative unit gate — ~5,400 tests
npm run test:e2e	Playwright end-to-end	Chromium is the release gate; full suite ~30 min
npm run test:e2e:onboarding	First-session flow	Onboarding-specific E2E
npm run test:e2e:accessibility	Axe a11y	Accessibility checks
npm run test:visual	Visual regression	Screenshot diffs

CI/CD & deployment

Deployment is **Vercel native git integration**: merging to `main` triggers a production build and deploy to the Sydney region. GitHub Actions runs 12 workflows on every PR — build, unit, E2E, accessibility, performance/Lighthouse, security, visual regression and compliance gates.

INHERITED CAVEAT · PRE-EXISTING RED GATES

Three CI gates are **red on `main` independent of any given change**: the GDPR and SOC2 compliance gates, and the live-Supabase RLS contract gate (which intermittently hits a statement timeout against the production DB). They are known and pre-existing — not regressions — but they make every PR look scarier than it is. The real go/no-go signal is the `qa-pipeline` unit gate plus the Chromium E2E gate. Cleaning these up is a tracked backlog item (Document 04).

Branch & release discipline

- Branch off `main`; open a PR; let CI run; merge on green (real gates).
- `main` is currently unprotected — adding branch protection once the red gates are fixed is recommended.
- Migrations are applied to the production Supabase project; keep `supabase/migrations/` the source of truth.
- Rollback: Vercel keeps prior production deployments as rollback candidates.



CTO HANDOVER PACK · 03

Workflow Mapping

The core execution flows of the platform — how a request travels, how a new tenant is born, how compliance posture is computed, how money is collected, and how code reaches production.

PREPARED
FOR
Farhad
Hussain
Incoming CTO

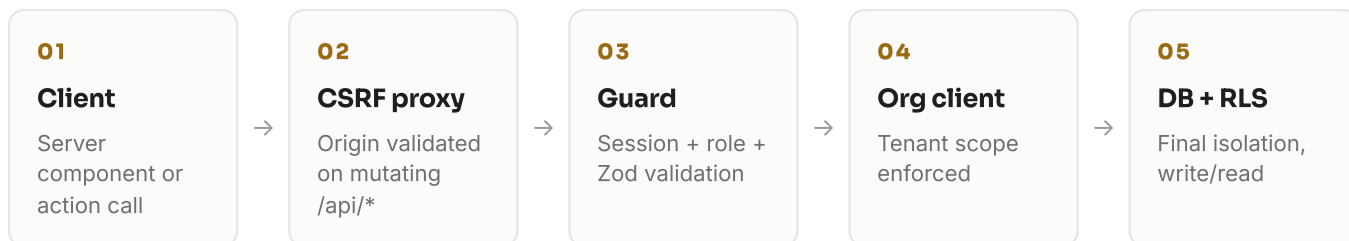
PREPARED
BY
Ejaz
Hussain
Founder

DATE
13 June
2026

CLASSIFICATION
Confidential

How a request travels

Every authenticated action passes through the same layered path. Two independent isolation layers (the org-scoped client and Postgres RLS) mean a single forgotten filter cannot leak another tenant's data.



☐ Application layer ☐ Security checkpoint ☐ Data layer

Why two layers

Server work sometimes uses the service-role key, which bypasses RLS. The org-scoped client guarantees the tenant filter in application code; RLS guarantees it at the database for session-key paths. Belt and braces — deliberately redundant because tenant isolation is the highest-severity property in a compliance product.

New tenant & first login

This is the birth of a customer — and where the current launch blocker lives (step 3).

○ Sign-up

app/(marketing) → /signin → Supabase Auth

User registers or accepts an invite; a Supabase auth user is created.

○ Session bootstrap

POST /api/auth/bootstrap

First authenticated call establishes the tracked session and kicks off workspace provisioning.

○ Tenant provisioning

lib/provisioning · recoverUserWorkspace

Creates the organisation, membership, subscription/trial, resolves industry framework packs and seeds onboarding status — ~30 sequential DB statements. THIS STEP CURRENTLY TIMES OUT (504) FOR NEW USERS UNDER LOAD.

○ Guided first session

/app · StartHereCard

Five guided actions (care plan → goal → progress note → evidence → review task); the dashboard stays focused to just the guide + live tiles until done.

○ Steady state

/app dashboard (command center)

Once the five steps complete (or an incident is logged), the full dashboard returns.

FIX DIRECTION

Collapse the provisioning chain into a single database-side transaction / RPC (round-trips are the multiplier), and/or move provisioning to an async 'setting up your workspace' step. Detailed in Document 04.

The compliance lifecycle

The core value loop: frameworks become controls, controls demand evidence, evidence and evaluations roll up into a live posture score and an audit trail.

○ Framework pack loaded

`framework-packs/*.json`

On tenant setup, industry-relevant packs (e.g. SOC 2, ISO 27001, NDIS) populate the control library.

○ Controls assigned

`lib/compliance, app/app/controls`

Controls get owners, cadence and status; obligations become tasks the team works.

○ Evidence captured

`app/app/vault, lib/evidence`

Documents, credentials and attestations are uploaded and mapped to controls.

○ Posture computed

`lib/compliance-graph.ts`

Controls + evidence + evaluations compute a readiness score and a node-wire compliance graph.

○ Surfaced & exported

`executive dashboard · audit packs`

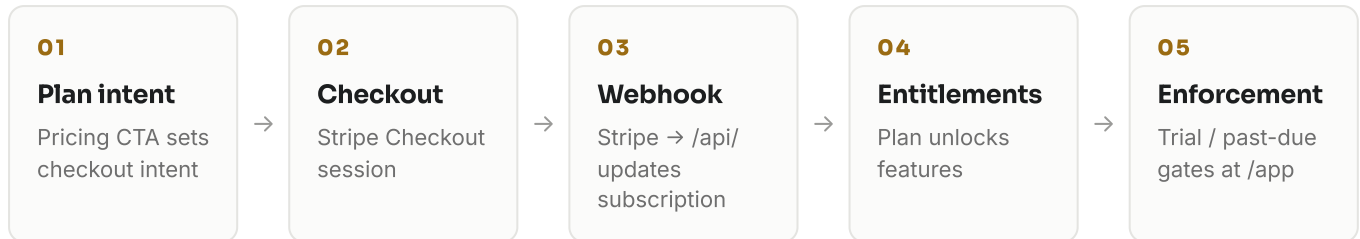
Posture shows on the executive view; audit-ready packs export for assessors.

○ Audit trail anchored

`trigger/ audit-chain-anchor`

Activity is written to a tamper-evident, periodically-anchored log.

Subscription & trial flow



Subscription state is webhook-driven and stored per organisation. The dashboard entry point (`app/app/page.tsx`) enforces it: a `pending_checkout` , `past_due` , `canceled` or `incomplete` status redirects the user to billing rather than the app. Plans map to entitlements that gate features across the product.

STATE	MEANING	USER EXPERIENCE
trialing	In free trial window	Full access; trial nudges shown
active	Paid & current	Full access
past_due	Payment failed	Redirected to billing
pending_checkout	Selected a plan, not paid	Routed into Stripe Checkout

Admin & support workflow

The admin console is the operator surface for running the platform without touching the database directly.

- **Customer-360** — a single org view with members, entitlements, support, activity, sessions, security and exports.
- **Reason-gated mutations** — admin changes require a stated reason and are written to the audit log.
- **Approval-gated high-risk changes** — delegated admins need approval for sensitive actions; founder access is unrestricted.
- **Live ops surfaces** — security, sessions and activity are observable in real time.
- **Immutable audit reads** — unified, legacy-inclusive audit history.

NOTE

The QA / audit runner and platform controls live here too — useful for you and the team to exercise flows safely against real tenants.

Release & deploy workflow

○ Branch

`git checkout -b ...`

Work happens on a feature branch off main.

○ Pull request

GitHub PR

Opening a PR triggers 12 CI workflows + a Vercel preview deployment.

○ CI gates

`build · unit · E2E · a11y · perf · security`

Merge on green real gates. Note the three pre-existing red gates (Doc 02 / Doc 04).

○ Merge to main

`squash / merge`

Triggers Vercel production build automatically.

○ Production deploy

Vercel · `syd1`

Live within minutes; prior deploys retained for rollback.

○ Verify

`prod smoke + runtime logs`

Confirm against production; check Sentry / Vercel runtime logs.



FORMA OS
COMPLIANCE, SIMPLIFIED

CTO HANDOVER PACK · 04

Project Plan & Roadmap

An honest read on launch-readiness: the one blocker and how to fix it, the known-issue backlog, a risk register, a phased roadmap, and a 30/60/90-day plan for your first quarter.

PREPARED FOR
Farhad Hussain
Incoming CTO

PREPARED BY
Ejaz Hussain
Founder

DATE
13 June 2026

CLASSIFICATION
Confidential

01 / 05

READINESS

Launch-readiness summary

Live PRODUCTION deployed & serving	1 LAUNCH BLOCKER provisioning timeout	3 RED CI GATES pre-existing	Done UX CYCLE elevation + first-run
---	--	--	--

The product is feature-complete for its target verticals and live in production. The visual quality and first-time-user experience were just overhauled. One database-performance blocker stands between today and a confident public launch; everything else is hardening.

Verdict

RECOMMENDATION

Do not stamp 'launch-ready' until a brand-new signup can reliably reach the dashboard. That is one focused engineering session (below). After it, the remaining items are quality/hardening that can run in parallel with a soft launch.

The launch blocker, in detail

SYMPTOM
New-user first login returns **504 (Vercel Runtime Timeout)** from `/api/auth/bootstrap` . Reproduced twice against production with the new-user critical-path smoke test; confirmed in production runtime logs.

Root cause

Bootstrap runs full tenant provisioning (`recoverUserWorkspace` → `ensureUserProvisioning` + a bootstrap RPC + framework resolution) as roughly 30 sequential database round-trips. The production database is slow enough per-statement that the chain exceeds the function timeout. Evidence from `pg_stat_statements` : the bootstrap RPC averages ~516 ms with spikes to ~8 s; a single `org_members` insert averages ~760 ms. The slowness is per-statement, so raising the function timeout alone (already done, PR 254) does not fix it — the round-trips are the multiplier.

Fix options (in recommended order)

OPTION	APPROACH	EFFORT / IMPACT
A · Single transaction	Collapse the provisioning chain into one DB-side function / transaction so it is one round-trip, not thirty	Med effort · highest impact
B · Profile per-statement cost	Investigate why a basic insert costs ~760ms — triggers, audit hooks, or RLS subqueries on write paths	Low-med · compounding wins
C · Compute tier	Confirm the Supabase compute tier is sized for production; upgrade if undersized	Low effort · quick relief
D · Async provisioning UX	Move provisioning off the request into a ‘setting up your workspace’ step (needs a small new UI)	Med · best UX, larger change

SUGGESTED PATH
Do C (immediate relief) + A (durable fix) together; use B to find any remaining per-statement tax. Reserve D if A/B/C don't fully clear it.

Known issues & hardening

ITEM	TYPE	PRIORITY
New-user provisioning timeout	Blocker	P0
Authenticated E2E suite has never had a full green run (server memory / heap-cap on <code>next start</code> ; DB saturation)	Reliability	P1
Three pre-existing red CI gates (GDPR, SOC2, live-RLS contract) — clean up or quarantine	CI hygiene	P1
Branch protection on <code>main</code> (currently unprotected)	Process	P1
Production DB performance review (indexing, triggers, compute)	Performance	P1
Secret rotation & ownership transfer across all systems	Security	P1
Fresh-eyes UX walkthrough of the newly-shipped product UI	Quality	P2

CONTEXT

The recent UX cycle (UI elevation, first-session focus, onboarding-bug fix, density passes) is **done and live** — it is no longer backlog. What remains is reliability, CI hygiene and the P0 blocker.

Risks & mitigations

RISK	IMPACT	LIKELIHOOD	MITIGATION
New users cannot complete first login	High	High (now)	Fix provisioning (P0) before any volume launch
Tenant data isolation regression	Critical	Low	Org-scoped client + RLS; register every new tenant table
Unprotected main → accidental bad merge	Med	Med	Add branch protection once red gates are fixed
Red CI gates mask a real failure	Med	Med	Quarantine known-red gates; keep real gates required
DB saturation under load	High	Med	Performance review; connection/timeout tuning
Knowledge concentration (lean team)	Med	High	This pack + onboarding you into all systems

Phased roadmap & first 90 days

Phased roadmap

PHASE	THEME	EXIT CRITERIA
Phase 0 — Launch readiness	Fix the P0 blocker; verify the new-user journey end to end	A fresh signup reaches the dashboard reliably; new-user E2E green
Phase 1 — Hardening	CI hygiene, branch protection, DB performance, secret rotation, full E2E green	All real gates green & required; clean PR signal
Phase 2 — Scale	Load testing, observability SLOs, cost & capacity review	Documented performance envelope; alerting in place

Your first 90 days

- **Days 0–30 · Land & unblock**
Get owner access to every system (Doc 01). Read Docs 02–03. Fix the P0 provisioning blocker and verify the new-user journey. Rotate secrets.
- **Days 30–60 · Harden**
Clear or quarantine the red CI gates, add branch protection, run the DB performance review, and get the authenticated E2E suite to a full green run.
- **Days 60–90 · Scale & own**
Load-test the provisioning + compliance paths, set observability SLOs/alerts, and form a view on team, architecture evolution and the next product bets.

CLOSING NOTE
FormaOS is further along than most products at this stage: live, multi-tenant, broad compliance coverage, and a product UI that now reads as enterprise. The honest gap is operational maturity — provisioning performance and CI discipline — which is exactly the kind of thing a CTO is brought in to own. This pack is the map; the codebase is the territory.